UNIT 4 - Question 2

Q) What is a function template? Explain with a general example.

Answer:

Function Template:

A function template in C++ is a generic function that works with different data types using the same code. Instead of writing separate functions for int, float, double, etc., we write one template function and the compiler creates the required function automatically based on the data type used.

Syntax:

```
template <class T>
T functionName(T a, T b)
{
    // function body
}
```

General Example:

Program to find the maximum of two values using a function template.

```
#include <iostream>
using namespace std;

template <class T>
T maximum(T a, T b)
{
    return (a > b) ? a : b;
}

int main()
{
    cout << maximum(10, 20) << endl;
    cout << maximum(4.5, 3.2) << endl;
    return 0;
}
```

Output:

```
20
4.5
```

Conclusion:

Thus, a function template helps us write reusable and type-independent functions in C++. It reduces code duplication and supports generic programming.

UNIT 4 - Question 3

Q) Explain overloading of template functions in C++ with a suitable example.

Ans: Function template overloading means defining more than one function template with the same name but different parameter lists. When a function call is made, the compiler selects the most suitable overloaded template based on the number and type of arguments. This gives flexibility and allows the same function name to work for different forms of data.

1. Need for overloading

1) It allows the same function name to be used for different kinds of operations.
2) It improves code reusability and readability.
3) The compiler chooses the best matching template automatically.

2. Syntax

```
template <class T>
return_type function_name(parameters);
```

We can define another overloaded template with different parameters.

3. Example

Program to overload template functions for finding maximum values.

```
#include <iostream>
using namespace std;

template <class T>
T maximum(T a, T b) {
    return (a > b) ? a : b;
}

template <class T>
T maximum(T a, T b, T c) {
    T m = (a > b) ? a : b;
    return (m > c) ? m : c;
}

int main() {
    cout << "Max of 10 and 20 = " << maximum(10, 20) << endl;
    cout << "Max of 3, 7 and 5 = " << maximum(3, 7, 5) << endl;
    cout << "Max of 2.5 and 1.8 = " << maximum(2.5, 1.8) << endl;
    return 0;
}
```

Output:

```
Max of 10 and 20 = 20
Max of 3, 7 and 5 = 7
Max of 2.5 and 1.8 = 2.5
```

4. Conclusion

Thus, overloaded template functions in C++ allow multiple templates with the same name but different parameter lists. They make generic programming more flexible and reduce repeated code.

UNIT 4 - Question 4

Q) What is template and implement bubble sort.

Answer:

1. Template

A template in C++ is a feature used for generic programming. It allows us to write a single function or class that can work with different data types. Instead of writing separate code for int, float, etc., we can use a template and reuse the same logic.

2. Bubble Sort

Bubble sort is a simple sorting technique in which adjacent elements are compared and swapped if they are in the wrong order. After each pass, the largest element moves to its correct position.

3. Syntax

```
template <class T>
return_type function_name(parameters);
```

4. Program to implement bubble sort using function template

```
#include <iostream>
using namespace std;

template <class T>
void bubbleSort(T a[], int n)
{
    for(int i = 0; i < n-1; i++)
    {
        for(int j = 0; j < n-i-1; j++)
        {
            if(a[j] > a[j+1])
            {
                T temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
    }
}

template <class T>
void display(T a[], int n)
{
    for(int i = 0; i < n; i++)
        cout << a[i] << " ";
}

int main()
{
    int a[] = {5, 2, 9, 1, 3};
    int n = 5;

    bubbleSort(a, n);

    cout << "Sorted array: ";
    display(a, n);

    return 0;
}
```

5. Output

```
Sorted array: 1 2 3 5 9
```

6. Conclusion

Thus, templates make programs reusable and type-independent. By using a function template, the same bubble sort logic can be used for different data types.

UNIT 4 - Question 5

Q) Differentiate between templates and macros in C++.

Answer:

Both templates and macros are used to write reusable code, but they work in different ways.

Point	Templates	Macros
1. Meaning :	Feature of C++ for generic programming.	Preprocessor directives using #define.
2. Type checking :	Type-safe and checked by compiler.	No type checking.
3. Working stage :	Processed during compilation.	Expanded before compilation.
4. Functionality :	Can be used for functions and classes.	Simple text substitution.
5. Debugging :	Easier to debug.	Harder to debug.
6. Safety and reliability :	Safer and more reliable.	May cause unexpected errors.

Example:

```
// Template example
 template <class T>
 T add(T a, T b) {
     return a + b;
 }

// Macro example
 #define ADD(a,b) a+b
```

Conclusion:

Templates are preferred in C++ because they are type-safe, flexible, and better for generic programming.
Macros are simpler but less safe.

UNIT 4 - Question 6

Q) Differentiate between templates and macros in C++.

Answer:

Both templates and macros are used to write reusable code, but they work in different ways.

Point	Templates	Macros
Meaning	Feature of C++ for generic programming.	Preprocessor directives using #define.
Type checking	Type-safe and checked by compiler.	No type checking.
Working stage	Processed during compilation.	Expanded before compilation.
Functionality	Can be used for functions and classes.	Simple text substitution.
Debugging	Easier to debug.	Harder to debug.
Safety and reliability	Safer and more reliable.	May cause unexpected errors.

Example:

```
// Template example
template <class T>
T add (T a, T b) {
    return a + b;
}

// Macro example
#define ADD(a,b) ((a) + (b))
```

Conclusion:

Templates are preferred in C++ because they are type-safe, flexible, and better for generic programming. Macros are simpler but less safe.